Bright Coffee Shop Sales

SQL Exercise: Wildcards & Window Functions

Databricks Hands-On Worksheet

Name: Thomo Mbatsane________________	**Due Date:** Tuesday, 19 May 2026	**Score:** ______ / 20

About this dataset
You are working with sales data from Bright Coffee Shop, a chain with three locations:
Lower Manhattan | Hell's Kitchen | Astoria

The table is called: coffee_sales

Columns: transaction_id, transaction_date, transaction_time, transaction_qty, store_id, store_location, product_id, unit_price, product_category, product_type, product_detail

Revenue = transaction_qty * unit_price (you will need to calculate this)

Instructions
1. Read each question carefully before writing or running any SQL.
2. Write your SQL in Databricks then copy it into the code box provided, you can use microsoft word or google document to fill out this document.
3. Answer the written questions in the yellow boxes - these do NOT require SQL.
4. If a question says [WRITTEN], no Databricks needed.
5. There are 20 marks in total. Each question shows its mark value.

6.
- Save a completed document as a pdf.
- upload the pdf document to GitHub
- Send your GitHub profile link, name, course name to (mvuko@brightlearn.co.za)

A

SQL Wildcards

LIKE, NOT LIKE, %, _, character ranges | Questions 1 - 8

Quick Reminder

% matches any sequence of characters (including none)
_ matches exactly one character
[a-z] matches any single character in a range (Databricks uses RLIKE for full regex)
LIKE is case-insensitive in most databases; NOT LIKE excludes matching rows

Question 1 [1 mark]

Find all products where the `product_detail` starts with the word 'Dark'.

Write a SELECT query using `LIKE` that returns `transaction_id`, `product_detail`, and `unit_price`.

```
SELECT  transaction_id,
        product_detail,
        unit_price
FROM    bright_coffee_shop_sales
WHERE   product_detail LIKE '%Dark%'
LIMIT   10;
```

How many distinct product_detail values start with 'Dark'? (run COUNT DISTINCT to find out)

```
SELECT count(DISTINCT Product_detail)
FROM bright_coffee_shop_sales
WHERE product_detail LIKE '%Dark%';
```

3

Question 2 [1 mark]

Find all transactions where `product_detail` ends with 'Lg' (large size).

Return `transaction_id`, `product_detail`, `product_category`, and `unit_price`. Order by `unit_price` descending.

```
SELECT  transaction_id,  product_detail,  product_category,  unit_price
FROM    bright_coffee_shop_sales
WHERE   product_detail LIKE '%Lg'
ORDER BY unit_price desc;
```

Question 3 [1 mark]

Find all products that contain the word 'Chai' anywhere in `product_type`.

Return `product_type` and `product_detail`. Use `DISTINCT` so each combination appears once.

```
SELECT  DISTINCT  product_type,
                  product_detail
FROM    bright_coffee_shop_sales
WHERE   product_detail LIKE '%Chai%'
ORDER BY  product_detail;
```

Question 4 [1 mark]

Use NOT LIKE to find all products that are NOT a type of tea.

Filter product_category so that it does NOT contain the word 'Tea'. Return DISTINCT product_category, product_type, and product_detail.

```
SELECT  DISTINCT  product_category,  product_type,  product_detail
FROM    bright_coffee_shop_sales
WHERE   product_category NOT LIKE '%TEA%'
ORDER BY  product_category,  product_type;
```

Question 5 [1 mark]

Use the underscore wildcard to find products with a 2-character size code at the end.

The size codes are 'Sm', 'Rg', and 'Lg'. These all follow a space and are exactly 2 characters. Match product_detail values that end with ' __' (space + any 2 chars). Return DISTINCT product_detail ordered alphabetically.

```
SELECT  DISTINCT  product_detail
FROM    bright_coffee_shop_sales
WHERE   product_detail LIKE '%_Sm' OR product_detail LIKE '%_Rg' OR product_detail LIKE '%_Lg'
ORDER BY  product_detail;
```

[WRITTEN] What is the difference between % and _ in a LIKE pattern? Give one example of each using this dataset.

Wildcard	Example	result	explanation
%	Where name like '%Chai%'	Morning sunrise chai	This wild card extracts any word which contains Chai
_	Where name like '_a%'	Cappuccino	Matches names where the letter is a.

Question 6 [1 mark]

Use RLIKE (regex) to find all products where product_detail contains a number.

Return DISTINCT product_detail. (Hint: the regex pattern for 'contains a digit' is [0-9].)

```
SELECT DISTINCT product_detail
FROM  bright_coffee_shop_sales
WHERE product_detail RLIKE '0-9'
ORDER BY product_detail;
```

Question 7 [2 marks]

Combine two wildcard conditions in one query.

Find transactions where product_category is 'Coffee' AND product_detail contains either 'Brazilian' or 'Ethiopian' (tip: use two OR conditions with LIKE).

Return transaction_id, transaction_date, store_location, product_detail, unit_price. Order by transaction_date.

```
SELECT  transaction_id,  transaction_date,  store_location,
        product_detail,  unit_price
FROM    bright_coffee_shop_sales
```

```sql
WHERE   product_category LIKE '%Coffee%'
  AND  product_detail LIKE '%Brazilian%'
        OR  product_detail LIKE '%Ethiopian%'
ORDER BY  transaction_date;
```

Question 8 [2 marks] [WRITTEN - no SQL needed]

Explain in your own words what the following pattern matches: product_detail LIKE '%Scone' Then write a different LIKE pattern that would match product names that contain the word 'Blend' anywhere in the middle of the name.

'%Scone'- Will return any word that ends with scone at the end.

'%Blend%'- Will return any word that contains Blend.

B	**Window Functions** Aggregate, Ranking, and Value functions | Questions 9 - 18

Quick Reminder

Syntax: function_name(column) OVER (PARTITION BY col ORDER BY col)

Aggregate: SUM, AVG, COUNT, MIN, MAX - calculate across the window without collapsing rows
Ranking: ROW_NUMBER, RANK, DENSE_RANK, NTILE(n) - assign position within a partition
Value: LAG(col, n), LEAD(col, n), FIRST_VALUE(col), LAST_VALUE(col) - access other rows

Part B1: Aggregate Window Functions

Question 9 [1 mark]

Calculate total revenue per store, keeping all individual rows.

Revenue = `transaction_qty * unit_price`. Use `SUM() OVER(PARTITION BY store_location)` to add a column called `store_total_revenue` to each row. Return `transaction_id`, `store_location`, `transaction_qty`, `unit_price`, and the new column. LIMIT 20.

```
SELECT
    transaction_id,
    store_location,
    transaction_qty,
    unit_price,
    SUM(transaction_qty * unit_price) OVER (PARTITION BY store_location)  AS  store_total_revenue
FROM    bright_coffee_shop_sales
LIMIT   20;
```

Looking at your results: does store_total_revenue change between rows in the same store? Why or why not?

We used the sum aggregate function which returns a total number of sales for the entire store not for a specific transaction.

Question 10 [1 mark]

For each transaction, show what percentage of its store's total revenue that transaction represents.

Calculate: `ROUND( (transaction_qty * unit_price) * 100.0 / SUM(transaction_qty * unit_price) OVER(PARTITION BY store_location), 2 )` as `pct_of_store_revenue`. Return `transaction_id`, `store_location`, revenue (calculated), and the percentage column. LIMIT 20.

```
SELECT
    transaction_id,
    store_location,
    ROUND(transaction_qty * unit_price, 2) AS  revenue,
    ROUND((transaction_qty * unit_price) * 100.00 / SUM(transaction_qty * unit_price) OVER
(PARTITION BY store_location) , 2) AS pct_of_store_revnue
FROM    bright_coffee_shop_sales
LIMIT   20;
```

Question 11 [1 mark]

Calculate a running total of revenue per store, ordered by transaction date and time.

Add `ORDER BY transaction_date, transaction_time` inside the OVER clause of a `SUM()` window. This gives a cumulative (running) total that increases with each transaction. Return `transaction_id`, `store_location`, `transaction_date`, `transaction_time`, `revenue`, `running_total`. LIMIT 30.

```
SELECT
    transaction_id,
    store_location,
    transaction_date,
    transaction_time,
    ROUND(transaction_qty * unit_price, 2)  AS  revenue,
    ROUND( SUM(transaction_qty * unit_price) OVER (ORDER BY transaction_date, transaction_time),
2)  AS  running_total
FROM    bright_coffee_shop_sales
LIMIT   30;
```

[WRITTEN] What is the key difference between the SUM() in Q9 (no ORDER BY) and the SUM() in Q11 (with ORDER BY)? What does adding ORDER BY change about how the window is calculated?

Sum() function without an order by clause calculates the entire data set where as if the order by function is included it will calculate according to the specified columns.

Part B2: Ranking Window Functions

Question 12 [1 mark]

Rank every transaction within its store by revenue (highest first).

Use `ROW_NUMBER()` with `PARTITION BY store_location` and `ORDER BY revenue DESC`. Return `transaction_id`, `store_location`, `revenue`, and `row_num`. LIMIT 30.

```
SELECT
    transaction_id,
    store_location,
    ROUND(transaction_qty * unit_price, 2)  AS revenue,
    ROW_NUMBER() OVER (PARTITION BY store_location ORDER BY revenue DESC)  AS  row_number
FROM    bright_coffee_shop_sales
ORDER BY  store_location,  row_number
LIMIT   30;
```

Question 13 [2 marks]

Compare RANK, DENSE_RANK, and ROW_NUMBER side by side on the same data.

Rank products by `unit_price` within each `product_category`, highest price first. Include all three functions as separate columns. LIMIT 40. Look for ties (same unit_price in same category) and observe how each function handles them.

```
SELECT
    product_category,
    product_detail,
    unit_price,
    ROW_NUMBER()  OVER( PARTITION BY product_category ORDER BY unit_price DESC)  AS row_number,
    RANK()        OVER( PARTITION BY product_category ORDER BY unit_price DESC )  AS rank,
    DENSE_RANK()  OVER( PARTITION BY product_category ORDER BY unit_price DESC )  AS dense_rnk
FROM    bright_coffee_shop_sales
ORDER BY  product_category,  unit_price  DESC
LIMIT   40;
```

[WRITTEN] Find a tie in your results (two rows with the same unit_price in the same category). Write down the values of rn, rnk, and d_rnk for those rows. Explain in one sentence why RANK and DENSE_RANK give the same result for tied rows but differ for the row after the tie.

Row 1 Rank 1 Dense_Rnk 1

Rank skips the next number to account for a tie and Dense rank groups the tie as 1 number and assigns the next number to the next value.

Question 14 [1 mark]

Use NTILE to divide transactions into 4 revenue quartiles within each store.

Calculate `revenue = transaction_qty * unit_price`. Then use `NTILE(4) OVER(PARTITION BY store_location ORDER BY revenue ASC)` to assign a quartile (1 = lowest, 4 = highest). Return `transaction_id`, `store_location`, `revenue`, `quartile`. LIMIT 30.

```
SELECT
    transaction_id,
    store_location,
    ROUND(transaction_qty * unit_price, 2)  AS  revenue,
    NTILE(4) OVER (ORDER BY transaction_qty * unit_price ASC) AS  quartile
FROM    bright_coffee_shop_sales
ORDER BY  store_location,  revenue  ASC
LIMIT   30;
```

Part B3: Value Window Functions

Question 15 [1 mark]

Use LAG to compare each transaction's revenue to the previous transaction in the same store.

Order by `transaction_date, transaction_time`. Use `LAG(revenue, 1, 0)` - the third argument `0` is the default value when there is no previous row. Return `transaction_id`, `store_location`, `transaction_date`, `revenue`, `prev_revenue`. LIMIT 20.

```
SELECT
    transaction_id,
```

```
    store_location,
    transaction_date,
    ROUND(transaction_qty * unit_price, 2)  AS  revenue,
    ROUND( LAG(transaction_qty * unit_price, 1, 0)  OVER(Order By transaction_date,
transaction_time), 2)  AS  prev_revenue
FROM   bright_coffee_shop_sales
LIMIT   20;
```

Question 16 [1 mark]

Use LEAD to show the next transaction's revenue alongside the current one.

Same partition and order as Q15. Use `LEAD(revenue, 1)`. Return `transaction_id`, `store_location`, `transaction_date`, `revenue`, `next_revenue`. LIMIT 20.

```
SELECT
    transaction_id,
    store_location,
    transaction_date,
    ROUND(transaction_qty * unit_price, 2)   AS  revenue,
    ROUND( LEAD(transaction_qty*unit_price, 1)  OVER(Order By transaction_date desc),
2)  AS  next_revenue
FROM    bright_coffee_shop_sales
LIMIT   20;
```

[WRITTEN] What value does next_revenue show for the very last transaction of each store partition? Why?

It shows resuts form the next row.

Question 17 [1 mark]

Use FIRST_VALUE to show the cheapest product in each category alongside every row.

Use `FIRST_VALUE(product_detail) OVER(PARTITION BY product_category ORDER BY unit_price ASC)`. Return `product_category`, `product_detail`, `unit_price`, and `cheapest_in_category`. Use DISTINCT to avoid too many duplicate rows. LIMIT 30.

```
SELECT  DISTINCT
    product_category,
    product_detail,
    unit_price,
    first_value(product_detail) OVER (PARTITION BY product_category ORDER BY unit_price ASC) AS
cheapest_in_category
FROM    bright_coffee_shop_sales
ORDER BY  product_category,  unit_price
LIMIT   30;
```

Question 18 [3 marks] - Challenge

This question combines wildcards AND window functions. Take your time - re-read the question before writing SQL.

Part A (1 mark): Filter the data to only include rows where `product_detail` ends in 'Lg' or 'Rg' (large or regular sizes). Use LIKE with an OR condition. Assign revenue as `transaction_qty * unit_price`.

Part B (1 mark): On that filtered data, use `RANK() OVER(PARTITION BY store_location ORDER BY revenue DESC)` to rank each transaction within its store.

Part C (1 mark): Also add the store's average revenue using `AVG() OVER(PARTITION BY store_location)` as `store_avg_revenue`.

Return: `transaction_id`, `store_location`, `product_detail`, `revenue`, `store_rank`, `store_avg_revenue`. LIMIT 40.

```
SELECT
    transaction_id,
    store_location,
    product_detail,
    ROUND(transaction_qty * unit_price, 2)   AS  revenue,
    RANK() OVER(PARTITION BY store_location ORDER BY transaction_qty * unit_price
DESC)   AS  store_rank,
    ROUND(AVG(transaction_qty * unit_price) Over(PARTITION BY
store_location),2)  AS  store_avg_revenue
FROM    Bright_Coffee_Shop_Sales
WHERE   product_detail LIKE '%Lg'
    OR  product_detail LIKE '%Rg'
ORDER BY  store_location,  store_rank
LIMIT   40;
```

[WRITTEN] Look at your results. Does every store_avg_revenue change between rows? Explain why or why not. Then explain in one sentence what store_rank = 1 means for each store.

The store average revenue is the same because the revenue amount is the same for the store we looking at.

Store_rank =1 means that each store is making the same amount of revenue so the system is grouping it together under the number 1 value.

End of Exercise - Well done!

Total marks: 20